

Motivation Efficient VLA inference without retraining



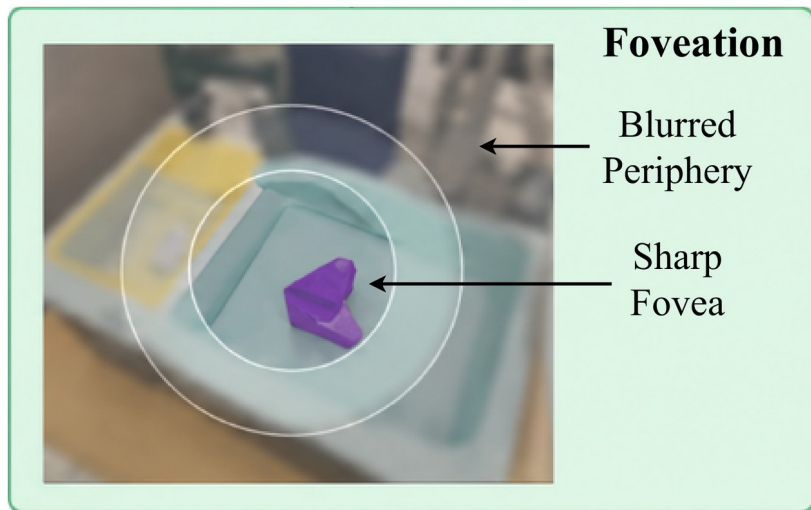
The control loop of a frozen VLA policy (Fig. 2 of the paper). Green, orange, purple, and blue boxes mark where the five tricks enter; Redundancy Control is action repeat.

[1] VLA inference is slow. Tens to hundreds of milliseconds per step (60 to 450 ms in our runs), which limits how often the policy can observe, decide, and correct.

[2] Gains are reported in isolation. Training-free tricks are typically evaluated for a specific trick, backbone, benchmark, and implementation, so transfer across models is unknown.

Do the gains transfer across VLA models and environments, **or are they configuration-dependent?**

Trick 1 Visual foveation



- Sharp central disc, progressively blurred periphery
- Keep ratio 20% or 50%; token count unchanged, no latency saving
- Tests whether high-frequency detail is needed only near the objects and the gripper

What to see

Original



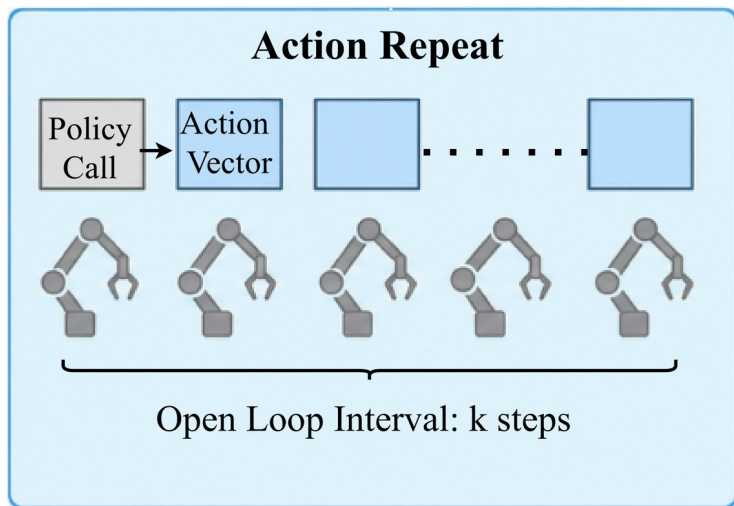
Foveation 20%



CogACT on WidowX, Eggplant in basket, same episode, 2x speed.
Green border = success, red = failure.

Trick one, visual foveation. It keeps a sharp central disc and blurs the periphery. Token count is unchanged, so it changes what the policy sees, not its cost.

Trick 2 Action repeat



- Hold each predicted action for $k = 2$ or 4 steps, about one call per k steps
- Cuts policy calls by about k , but lengthens the open-loop interval
- Fixed reduction, no state check: a speed-oriented baseline

When to act

Original



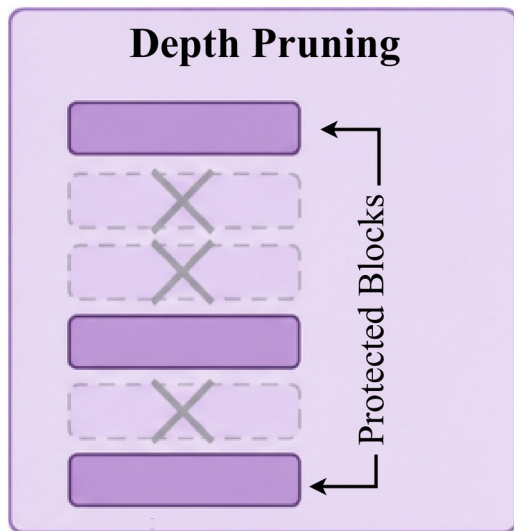
Action repeat 4



CogACT on WidowX, Carrot on plate, same episode, 2× speed.
Green border = success, red = failure.

Trick two, action repeat. Each action is held for two or four steps, so the policy is called less often. No state check, a pure speed baseline.

Trick 3 Depth pruning



- Score each decoder block by Block Influence
- Remove 1, 2, or 4 low-influence blocks; none adjacent, early blocks and the final block kept
- Fixed once from calibration data, not adapted at test time

How much to compute

Original



Depth pruning 2

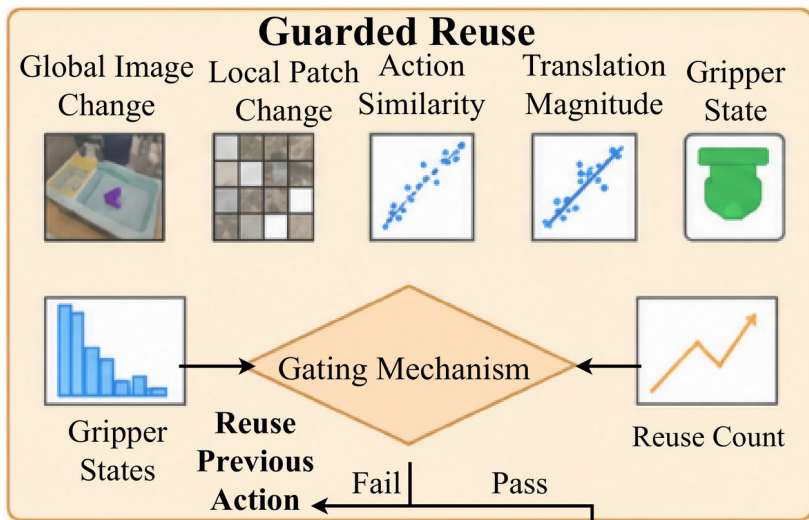


CogACT on WidowX, Carrot on plate, same episode, 2× speed.
Green border = success, red = failure.

Trick three, depth pruning. Decoder blocks that barely change their hidden states are removed, one, two, or four, keeping the early and final blocks. The selection is fixed before evaluation.

Trick 4 Guarded action reuse

When to act



- Reuse the previous action only when all gates pass: small global and local image change, recent actions agree, translation not near zero, gripper unchanged, reuse count below a cap
- Presets: strict, moderate, aggressive

Original



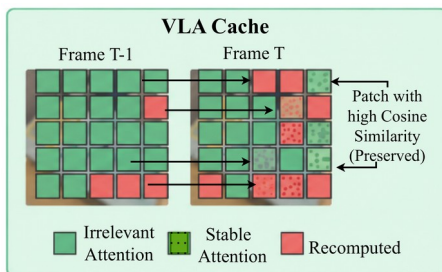
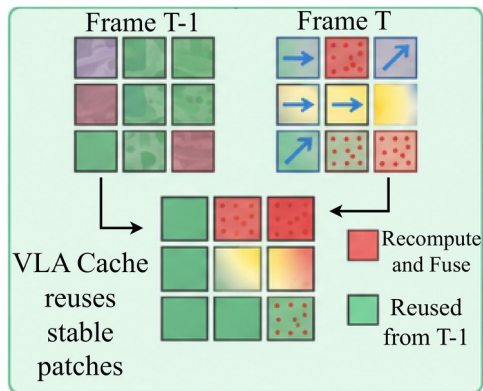
Reuse aggressive



CogACT on WidowX, Eggplant in basket, same episode, 2x speed. Green border = success, red = failure.

Trick four, guarded reuse. The previous action is reused only while every gate passes: image change, action agreement, translation, and gripper. Any failed gate restores a full call.

Trick 5 Temporal fusion



Top: fusion mask. Bottom: VLA-Cache criterion.

- Reuse visual tokens of stable patches from the previous call
- Recompute patches with motion, local structure, language attention, or in the interaction region
- Presets: motion-entropy, task-aware, conservative-adaptive

How much to compute

Original



Fusion motion



CogACT on WidowX, Spoon on towel, same episode, 2× speed.
Green border = success, red = failure.

Trick five, temporal fusion. Visual tokens of stable patches are reused from the previous call, while patches with motion, entropy, or language attention are recomputed. The policy still runs every step.

CogACT, WidowX: Eggplant in basket

All 14 configurations on the same episode (same initial state), 3× speed. Green border = success, red = failure.

Original



Foveation 20%



Foveation 50%



Action repeat 2



Action repeat 4



Depth pruning 1



Depth pruning 2



Depth pruning 4



Reuse strict



Reuse moderate



Reuse aggressive



Fusion motion



Fusion task-aware



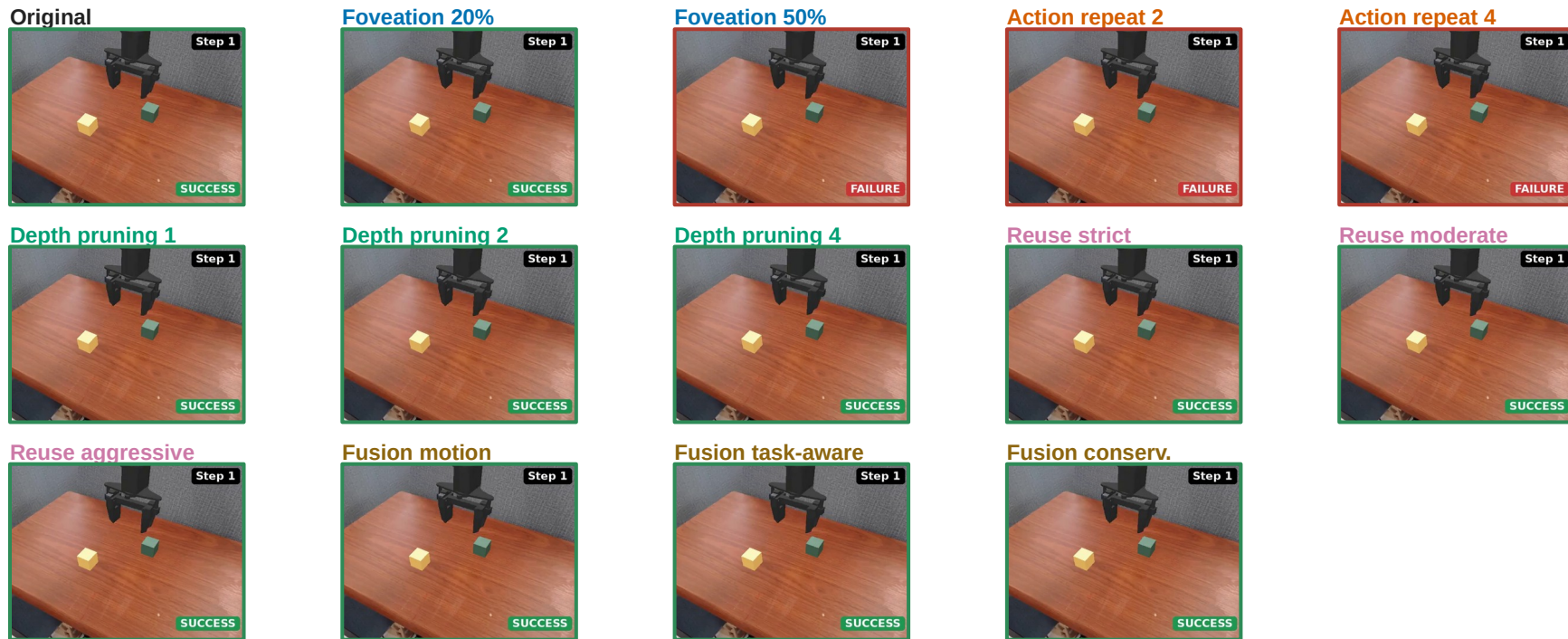
Fusion conserv.



Now every configuration on one episode. CogACT on WidowX, 12 of 14 succeed.

CogACT, WidowX: Stack cube

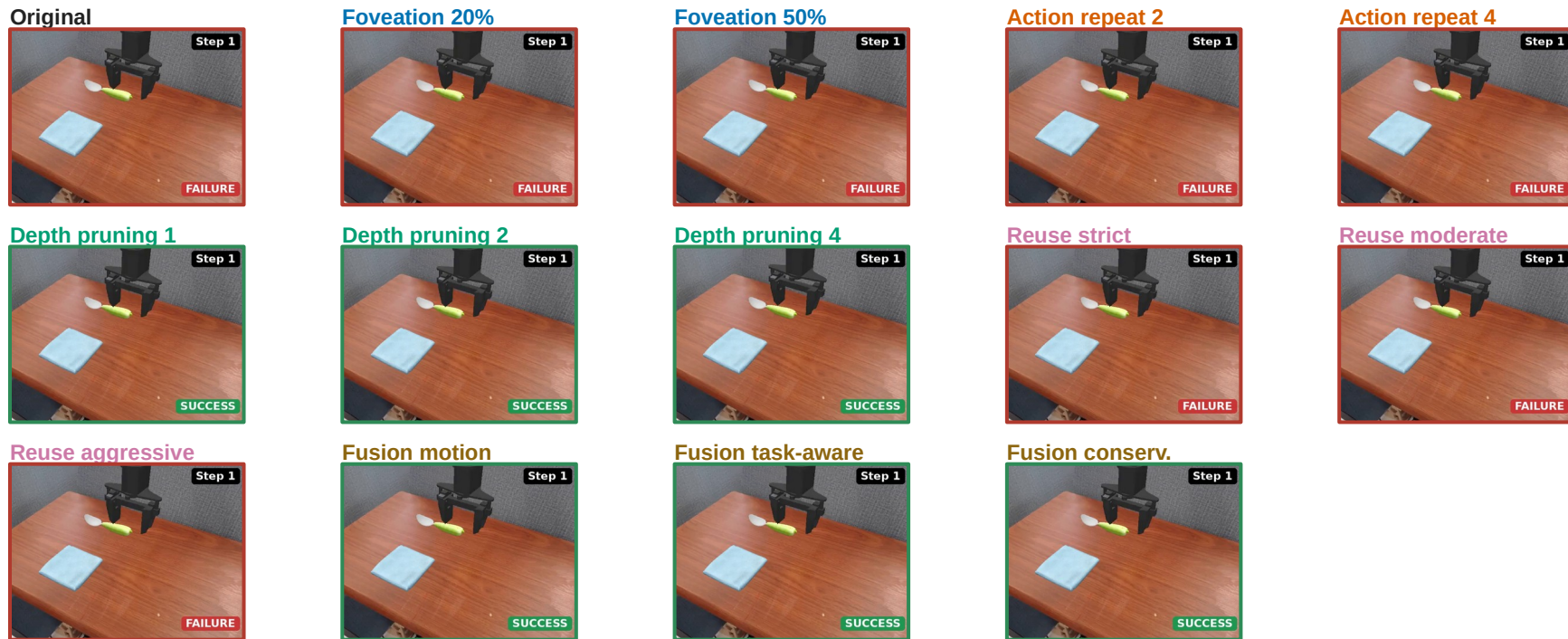
All 14 configurations on the same episode (same initial state), 2× speed. Green border = success, red = failure.



Stack cube: 11 of 14 succeed.

CogACT, WidowX: Spoon on towel

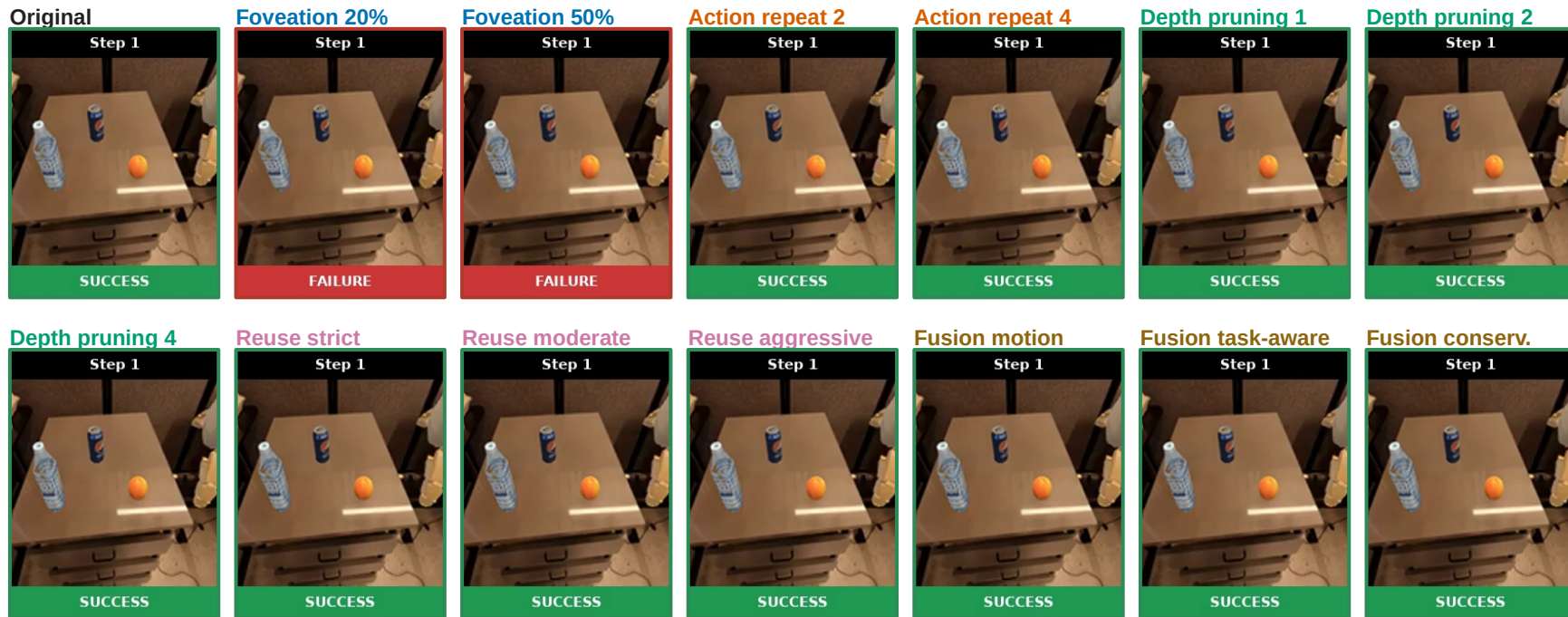
All 14 configurations on the same episode (same initial state), 2× speed. Green border = success, red = failure.



Spoon on towel: the original fails, 6 of 14 configurations succeed.

OpenVLA, Fractal: Move near

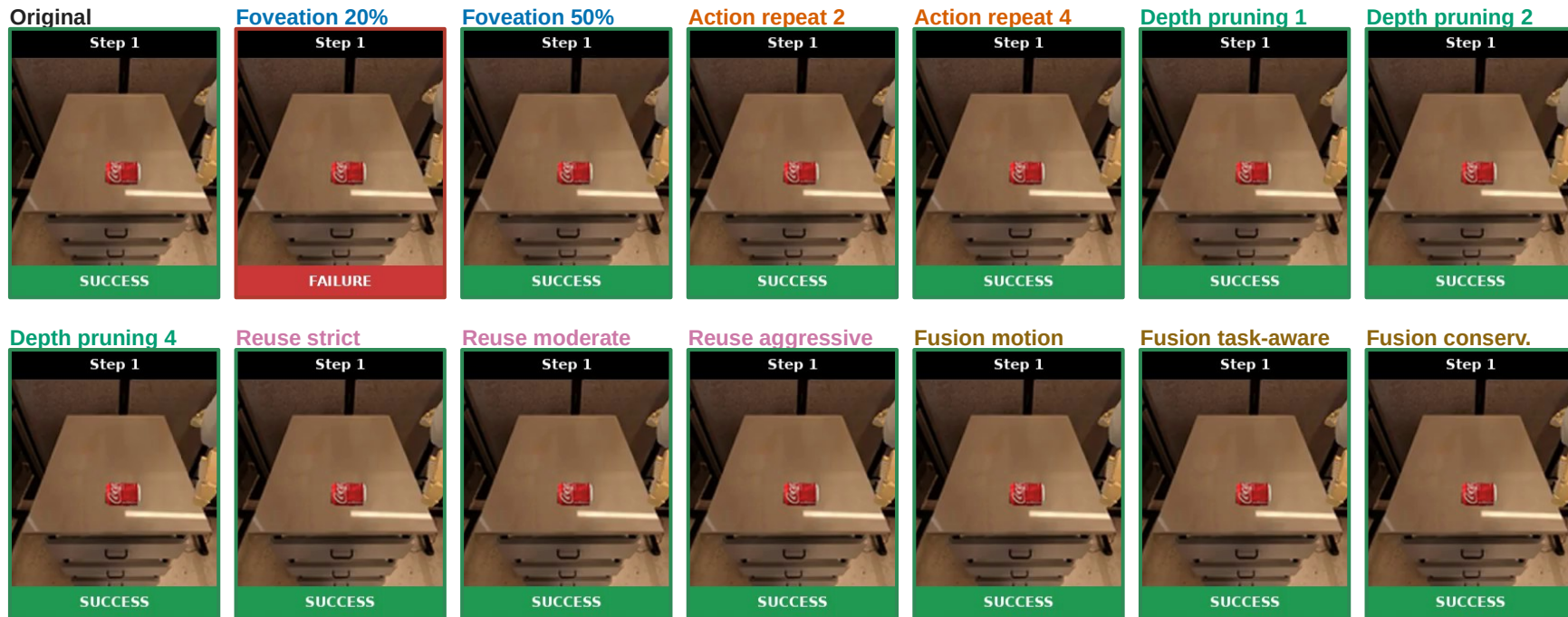
All 14 configurations on the same episode (same initial state), 2× speed. Green border = success, red = failure.



OpenVLA on Fractal, same episode for every configuration. Move near: 12 of 14 succeed, only foveation fails.

OpenVLA, Fractal: Pick coke can

All 14 configurations on the same episode (same initial state), 2× speed. Green border = success, red = failure.



Pick coke can: 13 of 14 succeed.

OpenVLA, Fractal: Open drawer

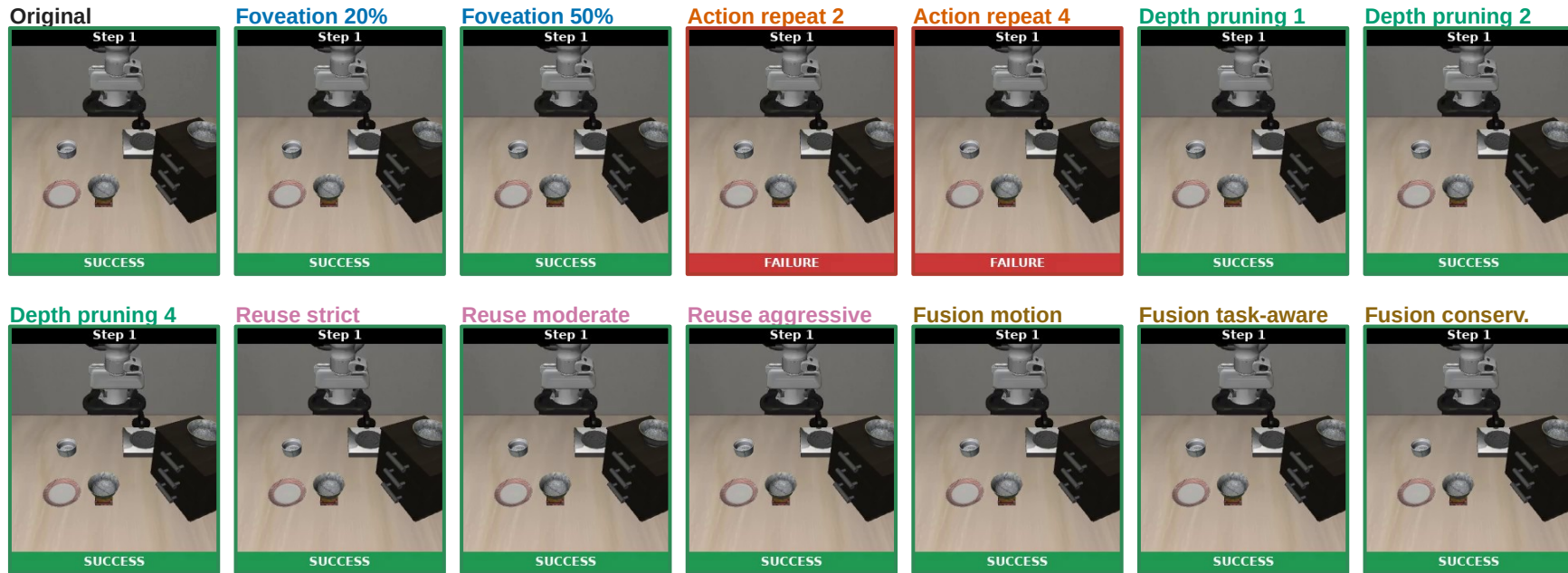
All 14 configurations on the same episode (same initial state), 2× speed. Green border = success, red = failure.



Open drawer: the original fails, 8 of 14 configurations succeed.

UniVLA, LIBERO Spatial: Bowl on cookie box to plate

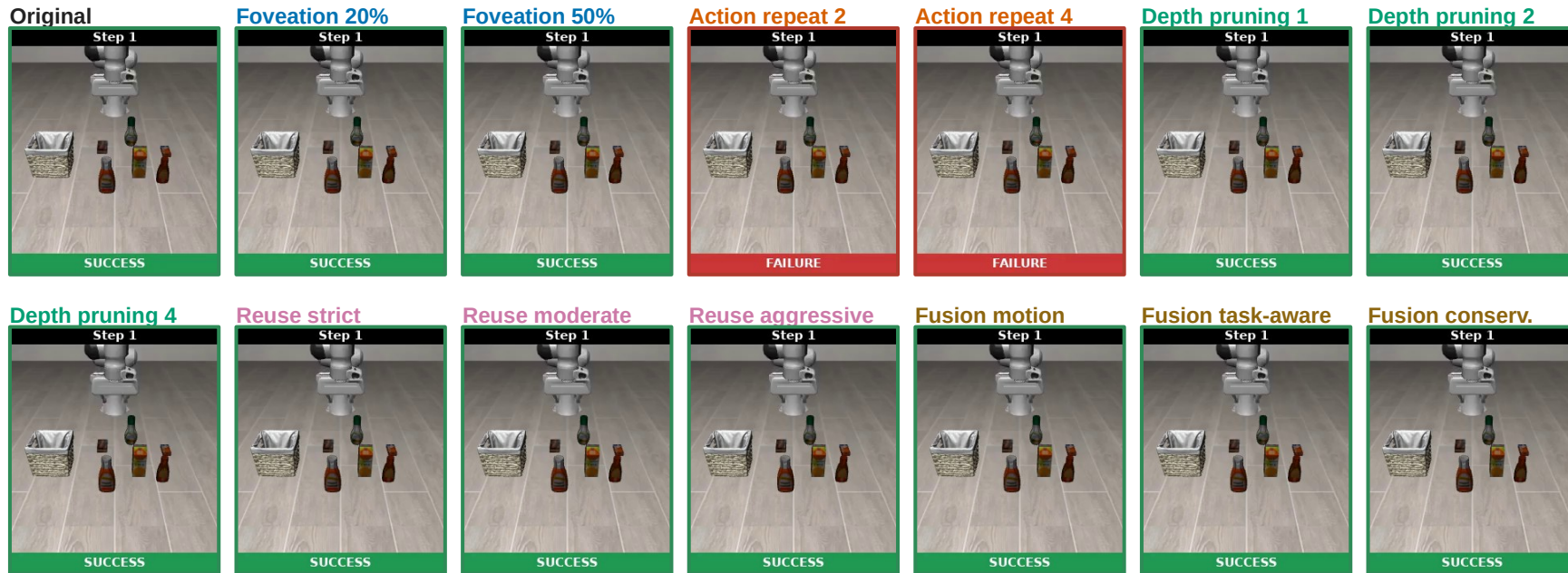
All 14 configurations on the same episode, 3× speed, runs cut at the slide end. Green border = success, red = failure.



UniVLA on LIBERO, four suites, same episode for every configuration. Spatial: 12 of 14 succeed.

UniVLA, LIBERO Object: Orange juice into basket

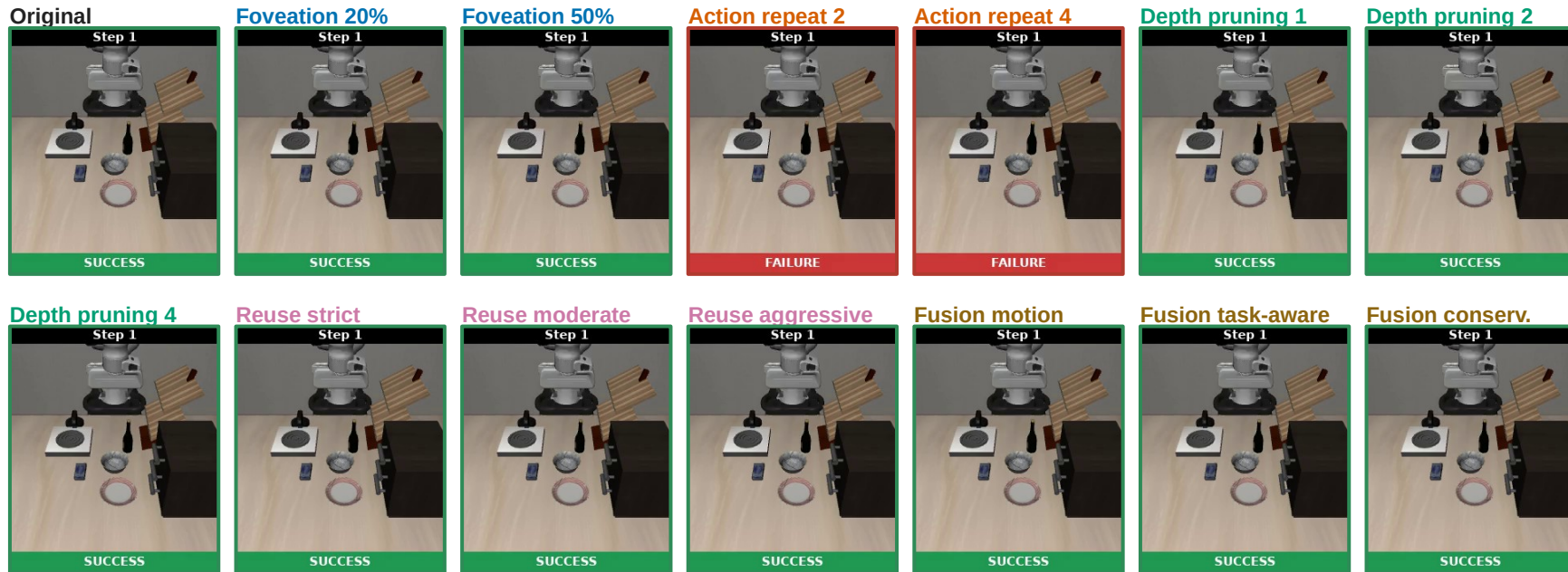
All 14 configurations on the same episode, 4× speed, runs cut at the slide end. Green border = success, red = failure.



Object: 12 of 14 succeed.

UniVLA, LIBERO Goal: Bowl on plate

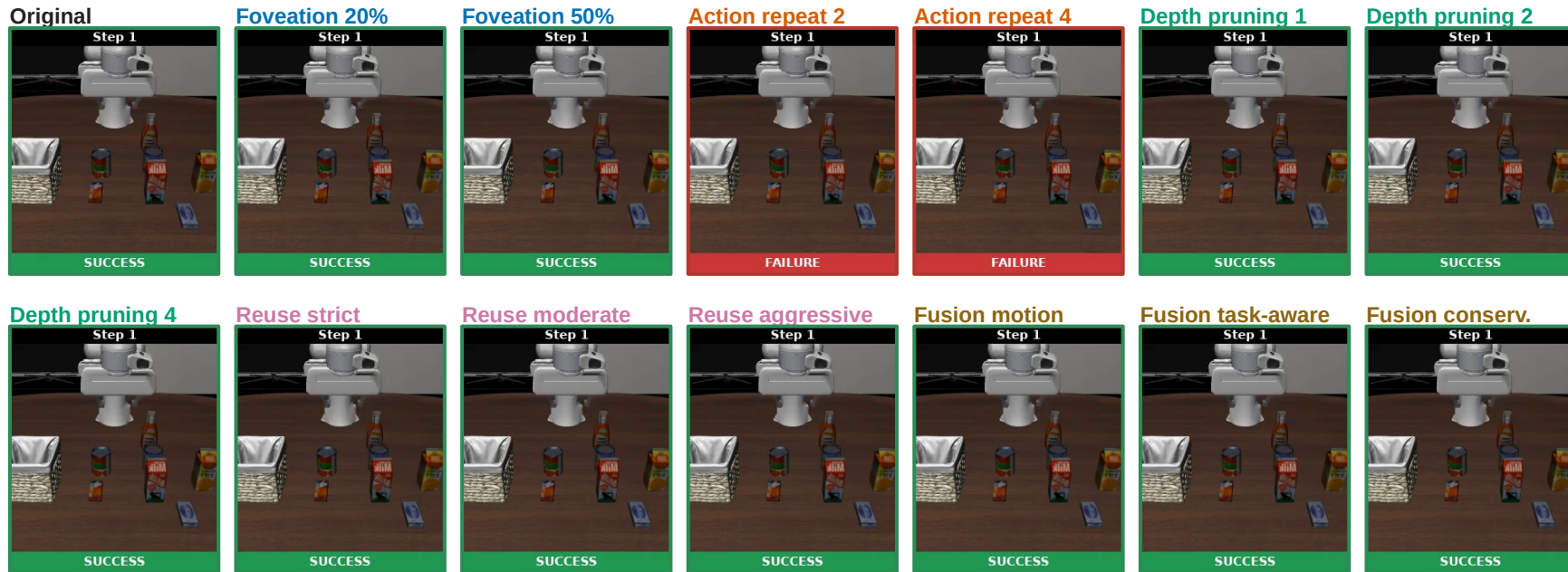
All 14 configurations on the same episode, 3× speed, runs cut at the slide end. Green border = success, red = failure.



Goal: 12 of 14 succeed.

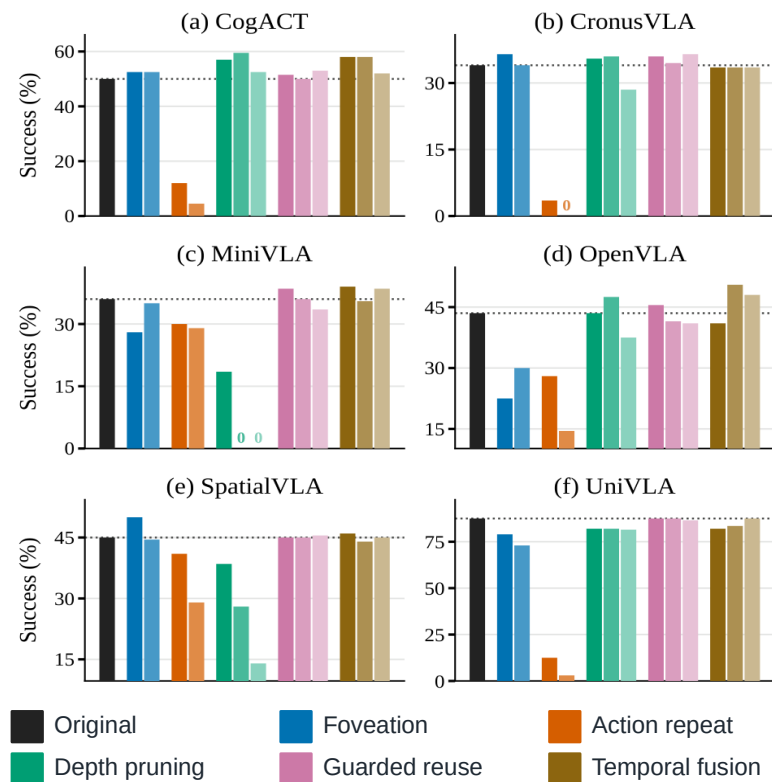
UniVLA, LIBERO Long: Cream cheese and butter into basket

All 14 configurations on the same episode, 8× speed, runs cut at the slide end. Green border = success, red = failure.



Long: 12 of 14 succeed. Only action repeat fails.

Which tricks preserve success?



Q1 Success

Guarded reuse and temporal fusion are the safest for preserving success.

Selected backbones also gain:

Foveation, WidowX: CogACT +2.5 · SpatialVLA +5.0 · CronusVLA +2.5

Depth pruning, WidowX: CogACT +9.5

Backbone-dependent:

Foveation, LIBERO Goal: UniVLA +7.0 vs SmoIVLA -13.0

Action repeat: mainly a speed baseline

Fig. 1: success per configuration, six WidowX backbones (14 bars each, darker to lighter within a trick). Dotted line = original policy, 0 = no successful episodes. Deltas from Tables I and II, best setting per trick.

Across seven backbones and three environments, guarded reuse and temporal fusion are the safest for preserving success. Foveation and depth pruning help selected backbones. Action repeat is mainly a speed baseline.

Which tricks are actually faster?

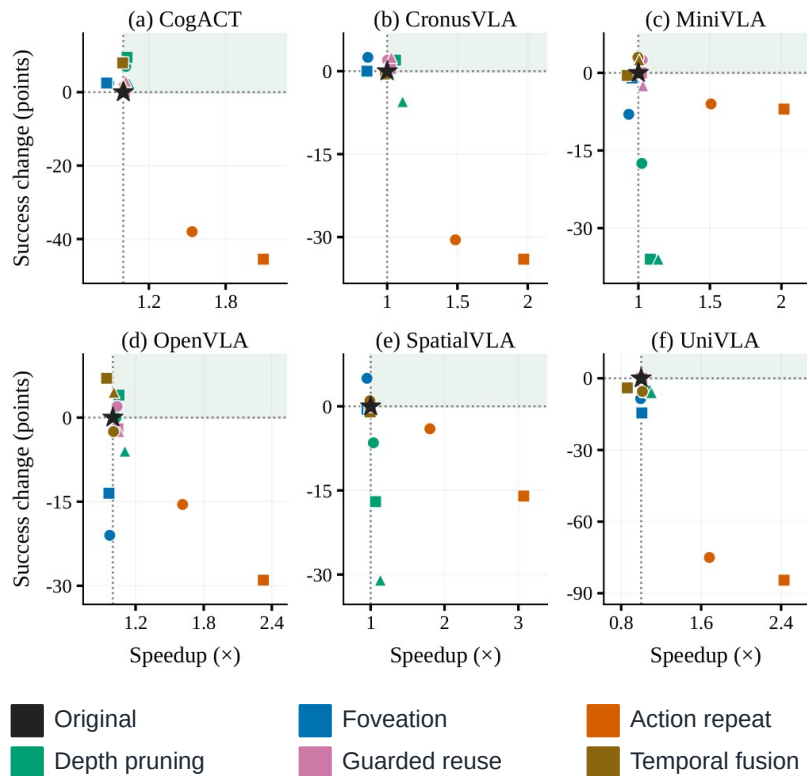


Fig. 4, SimplerEnv WidowX. Star = original, shaded = faster and no worse. Colour = trick; circle, square, triangle = first, second, third setting of that trick.

Q2 Speed

Action repeat gives the largest speedup, depth pruning gives decoder-level savings, guarded reuse gives the safest trade-off.

Latency, ms per step	Original → trick
Repeat · SpatialVLA WidowX	423.5 → 235.1
Repeat · OpenVLA LIBERO Long	171.6 → 93.2
Pruning · OpenVLA WidowX	210.2 → 199.4
Reuse · SmolVLA LIBERO Long	289.5 → 268.9

Second, speed. Action repeat gives the largest speedup. SpatialVLA on WidowX drops from about 424 to 235 milliseconds per step. Depth pruning gives smaller decoder-level savings, and guarded reuse the safest trade-off.

Takeaways

Guarded reuse

Most reliable. Preserves success, modest speedup when the gates open.

Action repeat

Largest speedup, but the risk of long open-loop execution.

Depth pruning

Decoder-level savings, benefit depends on the backbone.

Foveation · Temporal fusion

Test how much the visual representation can change while the policy stays stable.

VLA efficiency is policy- and environment-dependent. Evaluate every trick under the target backbone and environment before applying it.

The same trick helps one backbone and hurts another. Guarded reuse is most reliable, action repeat fastest but risky, depth pruning backbone-dependent. Evaluate every trick under the target backbone and environment. Thank you.